Name:Paleru Dharani Govardhan

Reg No : 192525280

TOPIC 5: GREEDY

1. There are 3n piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers piles where piles[i] is the number of coins in the ith pile. Return the maximum number of coins that you can have.

```python
def maxCoins(piles):
    piles.sort()
    n = len(piles)
    i = 0
    j = n - 2
    ans = 0

    for _ in range(n // 3):
        ans += piles[j]
        j -= 2
        i += 1

    return ans


# Test Case 1
piles = [2, 4, 1, 2, 7, 8]
print("Output:", maxCoins(piles))

# Test Case 2
piles = [2, 4, 5]
print("Output:", maxCoins(piles))
```

```
Output: 9
Output: 4
```

2. You are given a 0-indexed integer array coins, representing the values of thecoins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining

elements.

```python
def minPatches(coins, target):
    coins.sort()
    miss = 1
    i = 0
    count = 0

    while miss <= target:
        if i < len(coins) and coins[i] <= miss:
            miss += coins[i]
            i += 1
        else:
            miss += miss
            count += 1

    return count


# Test Case 1
coins = [1, 4, 10]
target = 19
print("Output:", minPatches(coins, target))

# Test Case 2
coins = [1, 4, 10, 5, 7, 19]
target = 19
print("Output:", minPatches(coins, target))
```

```
Output: 2
Output: 1
```

3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.

```
            job = jobs[index]

            for i in range(k):
                if workers[i] + job >= answer:
                    continue

                if i > 0 and workers[i] == workers[i - 1]:
                    continue

                workers[i] += job
                backtrack(index + 1)
                workers[i] -= job

                if workers[i] == 0:
                    break

        backtrack(0)
        return answer


    # Test Case 1
    jobs = [3, 2, 3]
    k = 3
    print("Output:", minimumTimeRequired(jobs, k))

    # Test Case 2
    jobs = [1, 2, 4, 7, 8]
    k = 2
    print("Output:", minimumTimeRequired(jobs, k))
```

```
Output: 3
Output: 11
```

4. We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.

```
        for i in range(1, n + 1):
            start, end, money = jobs[i - 1]

            # Find last job that ends <= current start
            j = bisect_right(end_times, start, 0, i - 1)

            take = money + dp[j]
            skip = dp[i - 1]

            dp[i] = max(take, skip)

        return dp[n]


    # Test Case 1
    startTime = [1, 2, 3, 3]
    endTime = [3, 4, 5, 6]
    profit = [50, 10, 40, 70]

    print("Output:", jobScheduling(startTime, endTime, profit))


    # Test Case 2
    startTime = [1, 2, 3, 4, 6]
    endTime = [3, 5, 10, 6, 9]
    profit = [20, 20, 100, 70, 60]

    print("Output:", jobScheduling(startTime, endTime, profit))
```

```
Output: 120
Output: 120
```

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where graph[i][j] denote the weight of the edge from vertex i to vertex j. If there is no edge between vertices i and j, the value is Infinity (or a very large number).

```
                new_dist = dist[u] + graph[u][v]

                if new_dist < dist[v]:
                    dist[v] = new_dist

        return dist


    # Test Case 1
    graph = [
        [0, 10, 3, float('inf'), float('inf')],
        [float('inf'), 0, 1, 2, float('inf')],
        [float('inf'), 4, 0, 8, 2],
        [float('inf'), float('inf'), float('inf'), 0, 7],
        [float('inf'), float('inf'), float('inf'), 9, 0]
    ]

    print("Output:", dijkstra_matrix(graph, 0))


    # Test Case 2
    graph = [
        [0, 5, float('inf'), 10],
        [float('inf'), 0, 3, float('inf')],
        [float('inf'), float('inf'), 0, 1],
        [float('inf'), float('inf'), float('inf'), 0]
    ]

    print("Output:", dijkstra_matrix(graph, 0))
```

```
Output: [0, 7, 3, 9, 5]
Output: [0, 5, 8, 9]
```

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.

```
    (0, 1, 7),
    (0, 2, 9),
    (0, 5, 14),
    (1, 2, 10),
    (1, 3, 15),
    (2, 3, 11),
    (2, 5, 2),
    (3, 4, 6),
    (4, 5, 9)
]

print("Output:", dijkstra_edges(6, edges, 0, 4))


# Test Case 2
edges = [
    (0, 1, 10),
    (0, 4, 3),
    (1, 2, 2),
    (1, 4, 4),
    (2, 3, 9),
    (3, 2, 7),
    (4, 1, 1),
    (4, 2, 8),
    (4, 3, 2)
]

print("Output:", dijkstra_edges(5, edges, 0, 3))
```

```
Output: 26
Output: 5
```

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

```
    codes = {}

    def generate(node, code):
        freq, chars, left, right = node

        if left is None and right is None:
            codes[chars] = code
            return

        generate(left, code + "0")
        generate(right, code + "1")

    generate(root, "")

    return [(ch, codes[ch]) for ch in characters]


# Test Case 1
characters = ['a', 'b', 'c', 'd']
frequencies = [5, 9, 12, 13]

print(huffmanCodes(characters, frequencies))


# Test Case 2
characters = ['f', 'e', 'd', 'c', 'b', 'a']
frequencies = [5, 9, 12, 13, 16, 45]

print(huffmanCodes(characters, frequencies))
```

```
[('a', '00'), ('b', '01'), ('c', '10'), ('d', '11')]
[('f', '1100'), ('e', '1101'), ('d', '100'), ('c', '101'), ('b', '111'), ('a', '0')]
```

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

```
[8]
✓ 0s
            else:
                current = current[3]

            # Leaf node
            if current[2] is None and current[3] is None:
                result.append(current[1])
                current = root

        return ''.join(result)


    # Test Case 1
    characters = ['a', 'b', 'c', 'd']
    frequencies = [5, 9, 12, 13]
    encoded_string = '1101100111110'

    print("Decoded:", decodeHuffman(
        characters, frequencies, encoded_string
    ))


    # Test Case 2
    characters = ['f', 'e', 'd', 'c', 'b', 'a']
    frequencies = [5, 9, 12, 13, 16, 45]
    encoded_string = '1100110111001011111001011'

    print("Decoded:", decodeHuffman(
        characters, frequencies, encoded_string
    ))

...  Decoded: dbcbdd
     Decoded: fefcbaac
```

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

[9]
✓ 0s

```python
def maximumWeight(weights, capacity):
    weights.sort(reverse=True)

    total = 0

    for weight in weights:
        if total + weight <= capacity:
            total += weight

    return total


# Test Case 1
weights = [10, 20, 30, 40, 50]
capacity = 60

print("Output:", maximumWeight(weights, capacity))


# Test Case 2
weights = [5, 10, 15, 20, 25, 30]
capacity = 50

print("Output:", maximumWeight(weights, capacity))
```

```
Output: 60
Output: 50
```

10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

[10]
✓ 0s

```python
def minimumContainers(weights, capacity):
    weights.sort(reverse=True)

    containers = 1
    current = 0

    for weight in weights:
        if current + weight <= capacity:
            current += weight
        else:
            containers += 1
            current = weight

    return containers


# Test Case 1
weights = [5, 10, 15, 20, 25, 30, 35]
capacity = 50

print("Output:", minimumContainers(weights, capacity))


# Test Case 2
weights = [10, 20, 30, 40, 50, 60, 70, 80]
capacity = 100

print("Output:", minimumContainers(weights, capacity))
```

```
Output: 4
Output: 5
```

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

```
        (0, 2, 6),
        (0, 3, 5),
        (1, 3, 15),
        (2, 3, 4)
    ]

    mst, total = kruskal(4, edges)

    print("Edges in MST:", mst)
    print("Total weight of MST:", total)


    # Test Case 2
    edges = [
        (0, 1, 2),
        (0, 3, 6),
        (1, 2, 3),
        (1, 3, 8),
        (1, 4, 5),
        (2, 4, 7),
        (3, 4, 9)
    ]

    mst, total = kruskal(5, edges)

    print("Edges in MST:", mst)
    print("Total weight of MST:", total)
```

```
Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19
Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]
Total weight of MST: 16
```

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.

[12]
✓ 0s

```
# Test Case 2
edges = [
    (0, 1, 1),
    (0, 2, 1),
    (1, 3, 2),
    (2, 3, 2),
    (3, 4, 3),
    (4, 2, 3)
]

given_mst = [
    (0, 1, 1),
    (0, 2, 1),
    (1, 3, 2),
    (3, 4, 3)
]

unique, mst, total = checkUniqueMST(5, edges, given_mst)

print("Is the given MST unique?", unique)
if not unique:
    print("Another possible MST:", mst)
print("Total weight of MST:", total)
```

```
Is the given MST unique? True
Total weight of MST: 19
Is the given MST unique? False
Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]
Total weight of MST: 7
```